TAC
Compilers

Consider the following syntax, which compared to the one presented in the lectures includes xors $B \oplus B$, implications $B \rightarrow B$, and switches $\texttt{switch}\ (B)\ S\ S$:

$$
\begin{array}{lcl}
P & ::= & S \\
S & ::= & S; S \mid [S] \mid id := E \mid \texttt{if}\ (B)\ S \mid \texttt{if}\ (B)\ S\ \texttt{else}\ S \mid \texttt{while}\ (B)\ S \mid \texttt{switch}\ (B)\ S\ S \\
E & ::= & E + E \mid E * E \mid num \mid id \\
B & ::= & \texttt{true} \mid \texttt{false} \mid E < E \mid E > E \mid E == E \mid E\ != E \\
  & \mid & B \mid\mid B \mid B\ \&\&\ B \mid !B \mid B \oplus B \mid B \rightarrow B
\end{array}
$$

As for an if-then-else statement, with a switch, if the Boolean expression is true, the then branch is executed, but as opposed to if-then-else statements, if the statement in the then branch updates the Boolean expression, making it false, the else branch is then executed.

1. Show how to translate xors to TAC.
2. Show how to translate implications to TAC.
3. Provide an example of a switch that executes both branches.
4. Show how to translate switch statements to TAC.

Solution

1. Xors can be translated as follows:

$$\begin{array}{lll}\texttt{tac}(B_1 \oplus B_2, L_1, L_2) & = & c_1 \parallel L' \parallel c_2 \parallel L'' \parallel c_3' \\ \quad \text{where } L' \text{ and } L'' \text{ are new labels} \\ \quad \text{and } \texttt{tac}(B_1, L', L'') = c_1 \\ \quad \text{and } \texttt{tac}(B_2, L_2, L_1) = c_2 \\ \quad \text{and } \texttt{tac}(B_2, L_1, L_2) = c_3 \end{array}$$

2. Implications can be translated as follows:

$$\begin{array}{lll}\texttt{tac}(B_1 \rightarrow B_2, L_1, L_2) & = & c_1 \parallel L' \parallel c_2 \\ \quad \text{where } L' \text{ is a new label} \\ \quad \text{and } \texttt{tac}(B_1, L', L_1) = c_1 \\ \quad \text{and } \texttt{tac}(B_2, L_1, L_2) = c_2 \end{array}$$

3. Here is an example where as a switch it runs both branches, and only the then branch would be run as an if-then-else:

$$id_1 := 0; id_2 := 1; \texttt{switch}\ (id_1 < id_2)\ (id_1 := 1)\ (id_1 := 0)$$

4. switches can be translated as follows:

$$\begin{array}{lll}\texttt{tac}(\texttt{switch}\ (B)\ S_1\ S_2, L) & = & c_1 \parallel L_1 \parallel \texttt{tac}(S_1, L_3) \parallel L_3 \parallel c_2 \parallel L_2 \parallel \texttt{tac}(S_2, L) \\ \quad \text{where } L_1, L_2, L_3 \text{ are new labels} \\ \quad \text{and } \texttt{tac}(B, L_1, L_2) = c_1 \\ \quad \text{and } \texttt{tac}(B, L, L_2) = c_2 \end{array}$$